

AIML320 Assignment 3 — Joshua Pinpin (ID: 300662880)

Part 1: Job Shop Scheduling

The instance has 3 jobs and 2 machines:

Job	Arrival	Op 1 (machine, time)	Op 2 (machine, time)
J1	0	O11 (M1, 50)	O12 (M2, 25)
J2	10	O21 (M2, 30)	O22 (M1, 35)
J3	20	O31 (M1, 40)	O32 (M2, 20)

Every schedule must satisfy three constraints: an operation O_{j1} cannot start before job J_j arrives (arrival), O_{j2} cannot start before O_{j1} has finished (precedence), and each machine processes at most one operation at a time (resource).

Q1. Earliest starting times (FCFS sequence)

The given sequence is:

Process(O11, M1, t_1) → Process(O21, M2, t_2) → Process(O31, M1, t_3) → Process(O12, M2, t_4) → Process(O22, M1, t_5) → Process(O32, M2, t_6)

This fixes the order of operations on each machine: M1 processes O11, then O31, then O22, and M2 processes O21, then O12, then O32. Following the hint, the earliest start time of each action is

- **ready time** of the operation = the job's arrival time (for O_{j1}), or the finish time of O_{j1} (for O_{j2}),
- **idle time** of the machine = the finish time of the previous operation on that machine (0 if none),
- **start** = max(ready time, idle time), and **finish** = start + processing time.

Working through the actions in order, and updating each machine's idle time after each step (both machines start idle at time 0):

#	Action	Proc time	Op ready time	Machine idle time	Start = max	Finish
1	O11 on M1	50	0 (J1 arrives)	M1: 0	$t_1 = 0$	50
2	O21 on M2	30	10 (J2 arrives)	M2: 0	$t_2 = 10$	40
3	O31 on M1	40	20 (J3 arrives)	M1: 50 (after O11)	$t_3 = 50$	90
4	O12 on M2	25	50 (O11 finishes)	M2: 40 (after O21)	$t_4 = 50$	75
5	O22 on M1	35	40 (O21 finishes)	M1: 90 (after O31)	$t_5 = 90$	125
6	O32 on M2	20	90 (O31 finishes)	M2: 75 (after O12)	$t_6 = 90$	110

Which constraint determines each start time:

1. **$t_1 = 0$** . J1 arrives at 0 and M1 is idle, so O11 starts immediately.
2. **$t_2 = 10$** . M2 is idle from time 0, but J2 only arrives at 10, so the arrival constraint is binding.

3. **t₃ = 50**. O31 is ready at 20 (J3's arrival), but M1 is busy with O11 until 50, so the resource constraint is binding.
4. **t₄ = 50**. M2 is free at 40 (after O21), but O12 cannot start until O11 finishes at 50, so the precedence constraint is binding.
5. **t₅ = 90**. O22 is ready at 40 (O21 has finished), but M1 is busy with O31 until 90, so the resource constraint is binding.
6. **t₆ = 90**. M2 is free at 75 (after O12), but O32 cannot start until O31 finishes at 90, so the precedence constraint is binding.

As a check, the start times satisfy $t_1 \leq t_2 \leq \dots \leq t_6$ ($0 \leq 10 \leq 50 \leq 50 \leq 90 \leq 90$), as required by the sorted sequence. The resulting schedule is:

Process(O11, M1, 0) → Process(O21, M2, 10) → Process(O31, M1, 50) → Process(O12, M2, 50) → Process(O22, M1, 90) → Process(O32, M2, 90)

Figure 1 shows this schedule as a Gantt chart. The waits identified above are visible as gaps: M2 is idle at 0–10 until J2 arrives, O31 waits for M1 until 50, and M2 is idle at 40–50 and 75–90 while O12 and O32 wait for their first operations to finish.

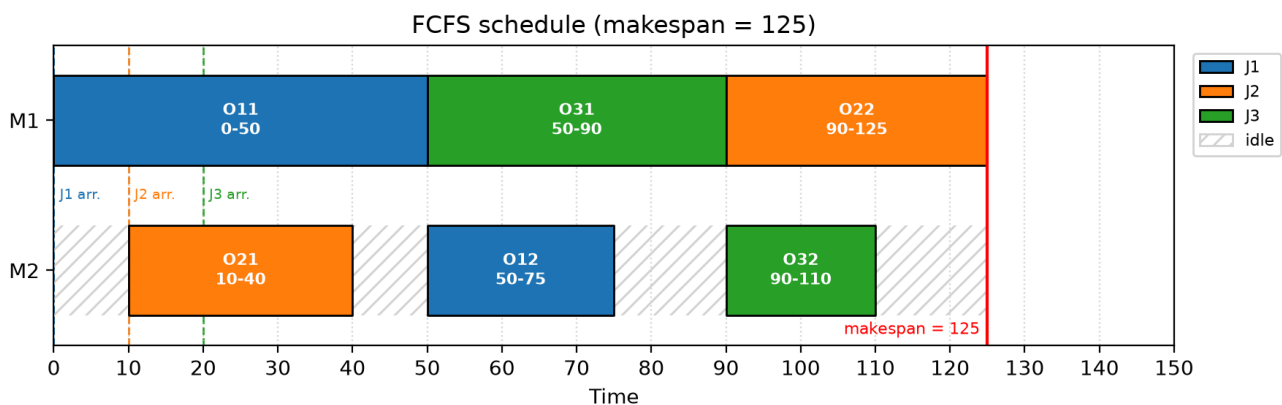


Figure 1: Gantt chart of the FCFS schedule from Q1 (makespan 125). Bars are coloured by job, hatched areas are machine idle time, and dashed lines mark job arrivals.

Q2. Completion times and makespan (FCFS)

The completion time of a job is the finish time of its last (second) operation, taken from the table in Q1:

Job	Last operation	Start	Proc time	Completion time
J1	O12	50	25	75
J2	O22	90	35	125
J3	O32	90	20	110

The makespan is the largest completion time over all jobs:

Makespan = max(75, 125, 110) = 125

J2 determines the makespan. Although J2 arrives second and its first operation O21 is finished by 40, its second operation O22 has to wait until 90 for M1, because the sequence puts O31 ahead of O22 on M1.

Q3. SPT schedule

A dispatching rule does not plan the whole schedule in advance. Instead, it simulates time moving forward and makes one decision at a time:

- Each machine has a **queue** of operations that are ready for it. An operation is ready when its job has arrived (for Oj1) or when Oj1 has finished (for Oj2).
- Whenever a machine is **idle** and its queue is **non-empty**, the rule chooses which queued operation to start next. The **Shortest Processing Time (SPT)** rule chooses the operation with the smallest processing time.
- A machine with an empty queue stays idle, even if an operation will become ready soon (non-delay dispatching).

The decision points are the times when a job arrives or an operation finishes, since these are the only times a queue or a machine's status can change. The trace below lists each decision point, with the processing times of queued operations in brackets.

Time t	Event(s)	M1 queue	M2 queue	Decision
0	J1 arrives → O11 ready	{O11(50)}	{}	M1 idle: starts O11, finishes at 50
10	J2 arrives → O21 ready	M1 busy	{O21(30)}	M2 idle: starts O21, finishes at 40
20	J3 arrives → O31 ready	{O31(40)} (M1 busy)	{}	M1 busy until 50: O31 waits
40	O21 finishes → O22 ready	{O31(40), O22(35)} (M1 busy)	{}	M2 idle but its queue is empty
50	O11 finishes → O12 ready	{O31(40), O22(35)}	{O12(25)}	M1: SPT picks O22 (35 < 40) , finishes at 85. M2: starts O12, finishes at 75
75	O12 finishes (J1 complete)	M1 busy	{}	M2 idle, queue empty
85	O22 finishes (J2 complete)	{O31(40)}	{}	M1 starts O31, finishes at 125
125	O31 finishes → O32 ready	{}	{O32(20)}	M2 starts O32, finishes at 145
145	O32 finishes (J3 complete)	{}	{}	All jobs done

The only real choice happens at **t = 50**. M1 becomes free, and two operations are waiting for it: O31 (ready since 20, 40 time units) and O22 (ready since 40, 35 time units). SPT picks O22 because it is shorter. FCFS would pick O31 because it has been waiting longer. Applying the same procedure with FCFS reproduces the Q1 schedule exactly, so this single decision is the only difference between the two solutions.

The final SPT solution, sorted by start time, is:

Process(O11, M1, 0) → Process(O21, M2, 10) → Process(O22, M1, 50) → Process(O12, M2, 50) → Process(O31, M1, 85) → Process(O32, M2, 125)

O22 and O12 both start at 50, so either order is valid. They are listed with M1 first, consistent with the tie at $t = 50$ in the Q1 sequence.

To double-check the hand calculations, I wrote a small optional script (`part1/gantt.py`). It checks both schedules against the arrival, precedence and resource constraints, and simulates non-delay dispatching with each rule. The simulation produces exactly the Q1 schedule when using FCFS and exactly the schedule above when using SPT. Figure 2 shows the SPT schedule.

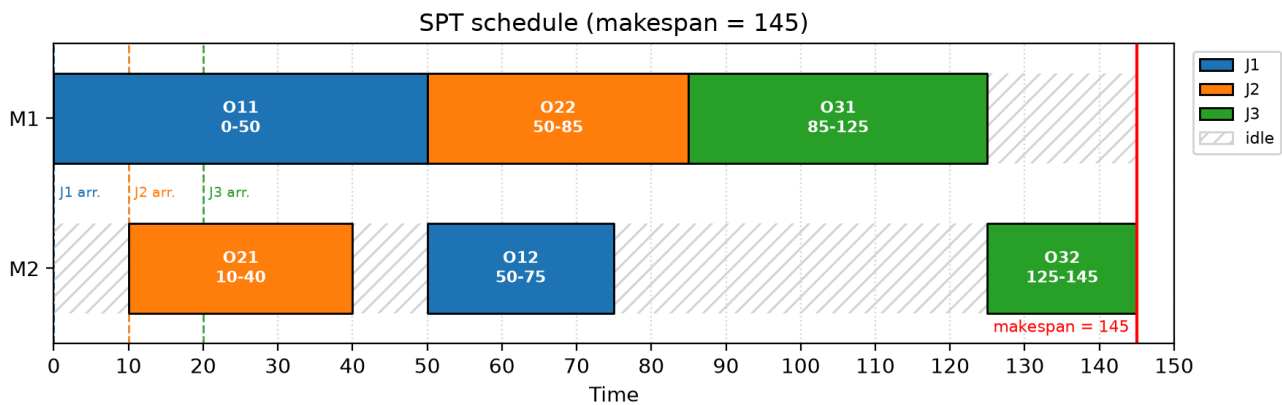


Figure 2: Gantt chart of the SPT schedule from Q3 (makespan 145), drawn on the same time axis as Figure 1. M2 is idle from 75 to 125 while it waits for O31 to finish on M1.

Q4. SPT completion times, makespan and comparison

As in Q2, each job's completion time is the finish time of its second operation:

Job	Last operation	Start	Proc time	Completion time
J1	O12	50	25	75
J2	O22	50	35	85
J3	O32	125	20	145

Makespan = $\max(75, 85, 145) = 145$

Comparison with the FCFS solution from Q1 and Q2:

	J1	J2	J3	Makespan
FCFS (Q1)	75	125	110	125
SPT (Q3)	75	85	145	145

In terms of makespan, the FCFS solution is better: 125 compared with 145, a difference of 20 time units.

SPT helps J2, which finishes 40 units earlier (85 instead of 125), but it delays J3 by 35 units (145 instead of 110). This happens because SPT runs O22 before O31 on M1, so O31 does not finish until 125. O32 cannot start on M2 until then, and M2 sits idle from 75 to 125. J3 now determines the makespan, and it is 20 units worse than under FCFS.

Q5. Does a better solution imply a better rule?

No. FCFS producing the better solution on this instance does not mean FCFS is a better rule than SPT in general, for three reasons.

1. "Better" depends on the objective. Makespan is only one way to measure a schedule. The table below compares both solutions on two other common objectives: mean flowtime (the average time a job spends in the shop, completion time minus arrival time) and total completion time. The values were computed from the completion times in Q2 and Q4.

Rule	Completion (J1, J2, J3)	Flowtime (J1, J2, J3)	Mean flowtime	Total completion time	Makespan
FCFS	75, 125, 110	75, 115, 90	93.33	310	125
SPT	75, 85, 145	75, 75, 125	91.67	305	145

SPT is worse on makespan but better on both flowtime-based objectives, so even on this one instance neither rule dominates the other.

2. One small instance is not enough evidence. This instance has only 3 jobs and 2 machines, and the two solutions differ in a single decision (at $t = 50$). Dispatching rules are heuristics, and how well they perform varies a lot between instances. Small changes can flip or remove the difference. For example, if J3 arrived after $t = 50$, O31 would not be in M1's queue at that decision point, and both rules would produce the same schedule. To conclude that one rule is better, both rules should be tested on many instances (e.g. randomly generated instances or a standard benchmark set), and their average performance should be compared, ideally with a statistical significance test.

3. Each rule suits different goals. SPT is greedy and myopic. At $t = 50$ it chose O22 (35) over O31 (40) because O22 is shorter, without considering that J3 still had 60 units of work left (O31 + O32) while J2 had only 35. Delaying J3 left M2 idle from 75 to 125 and increased the makespan. A rule that considers remaining work, such as Most Work Remaining (MWKR), would have chosen O31 here and is generally better suited to makespan. SPT, by finishing short operations first, tends to reduce flowtime-type objectives, which is what happened here. So which rule is "better" depends on what the scheduler is trying to optimise.

Q6. (AIML420 only)

Q6 is required for AIML420 students only. This report is for AIML320, so Q6 was not attempted.

Part 2: Neural Networks

Part 2 compares a single perceptron, which I implemented from scratch (`part2/perceptron.py`), with a library multilayer perceptron (`part2/mlp.py`). Task a trains the perceptron on SeaSyn, Task b trains the same perceptron on RingSyn, and Task c trains an MLP on RingSyn. Accuracy (the fraction of test samples classified correctly) is the metric throughout. Every table and figure in this part was produced by the submitted programs, using the commands in `readme.txt`. A few supporting checks mentioned in the text (such as the linear separability test in Task a) were run separately, and are noted where they appear.

Data

Part 2 uses two binary classification datasets. Each has a separate train and test file, and the target column is `class` (0 or 1).

Dataset	Train rows	Test rows	Features	Class 1 share (train / test)
SeaSyn	100	40	<code>attrib1</code> , <code>attrib2</code> , <code>attrib3</code> (values between 0 and 10)	52.0% / 52.5%
RingSyn	453	300	<code>feature1</code> , <code>feature2</code> (values between about -1.2 and 1.1)	66.9% / 65.7%

Both datasets are close enough to balanced for accuracy to be a sensible metric, although RingSyn leans towards class 1. A model that always predicts class 1 already gets **65.7%** on the RingSyn test set, so this majority-class baseline is used as the yardstick in Tasks b and c. A useful model has to beat it clearly.

In RingSyn, class 1 forms a complete ring around the origin (most points are 0.54–0.88 from the origin), and class 0 forms a wider half-ring outside it (0.82–1.15 from the origin) that only covers the lower half: every class 0 point has `feature2` < 0. The two classes overlap slightly where their distances from the origin meet, at about 0.85, and they cannot be separated by a straight line (see Figure 6). The handout calls the RingSyn features `x1` and `x2`, but the files name them `feature1` and `feature2`, so the code reads the column names from each file's header instead of hard-coding them.

One thing to note about SeaSyn: the 40 test rows are identical to the first 40 rows of the training file. The SeaSyn "test" accuracy in Task a is therefore measured on data the perceptron was trained on, and it shows how well the model fits the training data rather than how well it generalises. The RingSyn train and test files share no rows.

Perceptron implementation

The perceptron used in Tasks a and b is written from scratch in `part2/perceptron.py`, using only numpy for array maths. The same class is used, unchanged, for both tasks.

Part	Choice	Reason
Activation	Step function: $\hat{y} = 1$ if $w \cdot x + b \geq 0$, otherwise $\hat{y} = 0$	This is the classic Rosenblatt perceptron, and its 0/1 output matches the dataset labels.
Bias	A separate bias term b , updated like a weight on a constant input of 1	Without it, the decision boundary would have to pass through the origin.
Initialisation	$w = 0$ and $b = 0$	Deterministic, so results only depend on the shuffle seed.
Learning rule	Online (one sample at a time): $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$	The standard perceptron rule. $y - \hat{y}$ is 0 when the prediction is correct, so the weights only change on mistakes. A false negative ($y = 1, \hat{y} = 0$) moves the boundary towards x , and a false positive moves it away.
Learning rate	$\eta = 1.0$	Because the weights start at zero, every update is a multiple of η , so η only scales w and b and never changes which side of the boundary a point falls on. Any positive η gives exactly the same predictions, and I confirmed this by training with $\eta = 0.01, 1$ and 7 .
Epoch	One full pass over the training set, in a new random order each epoch	Shuffling stops the result depending on the order of rows in the file. The shuffle uses a fixed seed (42), so every run is reproducible.
Feature scaling	Each feature is standardised to mean 0 and standard deviation 1, using the mean and std of the training set only, and the same values are applied to the test set	SeaSyn features range from 0 to 10, while the bias input is always 1. Without scaling, the bias would learn much more slowly than the weights. Fitting the scaler only on the training set prevents test data leaking into training.

During training the model records how many training samples it misclassified (and therefore updated on) in each epoch. If this count reaches 0, the boundary separates the training data perfectly and no further epoch changes the weights. If it never reaches 0, the boundary keeps moving every epoch.

To check the implementation before using it on the assignment data, `python3 part2/perceptron.py --selftest` trains the perceptron for 20 epochs on two 4-point logic gates:

- **AND** (linearly separable): the error count goes 1, 3, 2, 2, 2, 3 and then stays at 0 from epoch 8, ending with 100% accuracy.
- **XOR** (not linearly separable): there is at least one error in every epoch and accuracy ends at 50%, because no single line can separate XOR's classes.

Task a: Perceptron on linearly separable data (SeaSyn)

For each number of epochs in the handout's list, a fresh perceptron was trained on SeaSynTrain and evaluated on SeaSynTest (`python3 part2/perceptron.py --task a`). Every model uses the same shuffle seed (42), so the model trained for E epochs is exactly the first E epochs of the 200-epoch model, and the table shows snapshots of a single training run. The last column is the number of training samples the perceptron got wrong (and updated on) during its final epoch.

Model (epoch)	Train accuracy	Test accuracy	Mistakes in last epoch
Perceptron (1)	0.630	0.675	31
Perceptron (5)	0.750	0.700	26
Perceptron (10)	0.740	0.675	25
Perceptron (15)	0.560	0.550	24
Perceptron (20)	0.840	0.850	23
Perceptron (50)	0.660	0.550	30
Perceptron (60)	0.730	0.750	25
Perceptron (80)	0.780	0.750	19
Perceptron (100)	0.750	0.700	18
Perceptron (120)	0.750	0.775	25
Perceptron (150)	0.480	0.400	20
Perceptron (200)	0.740	0.775	29

One seed could be misleading, so the sweep was repeated with seeds 0–9. The table below gives the mean $\pm$ standard deviation of test accuracy. It also shows a "pocket" version of the same perceptron. Its learning is identical, but at the end of each epoch it measures training accuracy and keeps the best weights seen so far, instead of the weights left by the last epoch. The pocket version only uses the training data to choose its weights.

Epochs	Perceptron test accuracy	Pocket perceptron test accuracy
1	0.745 $\pm$ 0.073	0.745 $\pm$ 0.073
5	0.738 $\pm$ 0.065	0.823 $\pm$ 0.024
10	0.773 $\pm$ 0.045	0.827 $\pm$ 0.026
15	0.807 $\pm$ 0.059	0.847 $\pm$ 0.021
20	0.733 $\pm$ 0.056	0.847 $\pm$ 0.024
50	0.742 $\pm$ 0.084	0.853 $\pm$ 0.018
60	0.753 $\pm$ 0.062	0.853 $\pm$ 0.018
80	0.688 $\pm$ 0.103	0.850 $\pm$ 0.016
100	0.722 $\pm$ 0.124	0.855 $\pm$ 0.015
120	0.742 $\pm$ 0.089	0.855 $\pm$ 0.015
150	0.703 $\pm$ 0.121	0.855 $\pm$ 0.015
200	0.747 $\pm$ 0.064	0.857 $\pm$ 0.016

Task a: SeaSyn

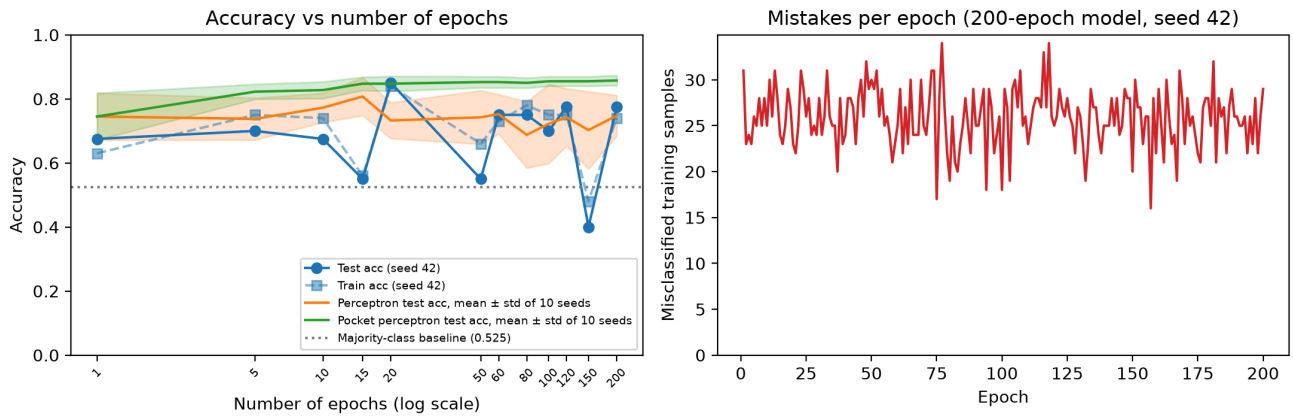


Figure 3: Left: accuracy against the number of epochs (log scale) on SeaSyn, for seed 42 and as the mean $\pm$ std over seeds 0–9 for the plain and pocket perceptrons. Right: training samples misclassified in each epoch of the 200-epoch model (seed 42). The count never reaches 0.

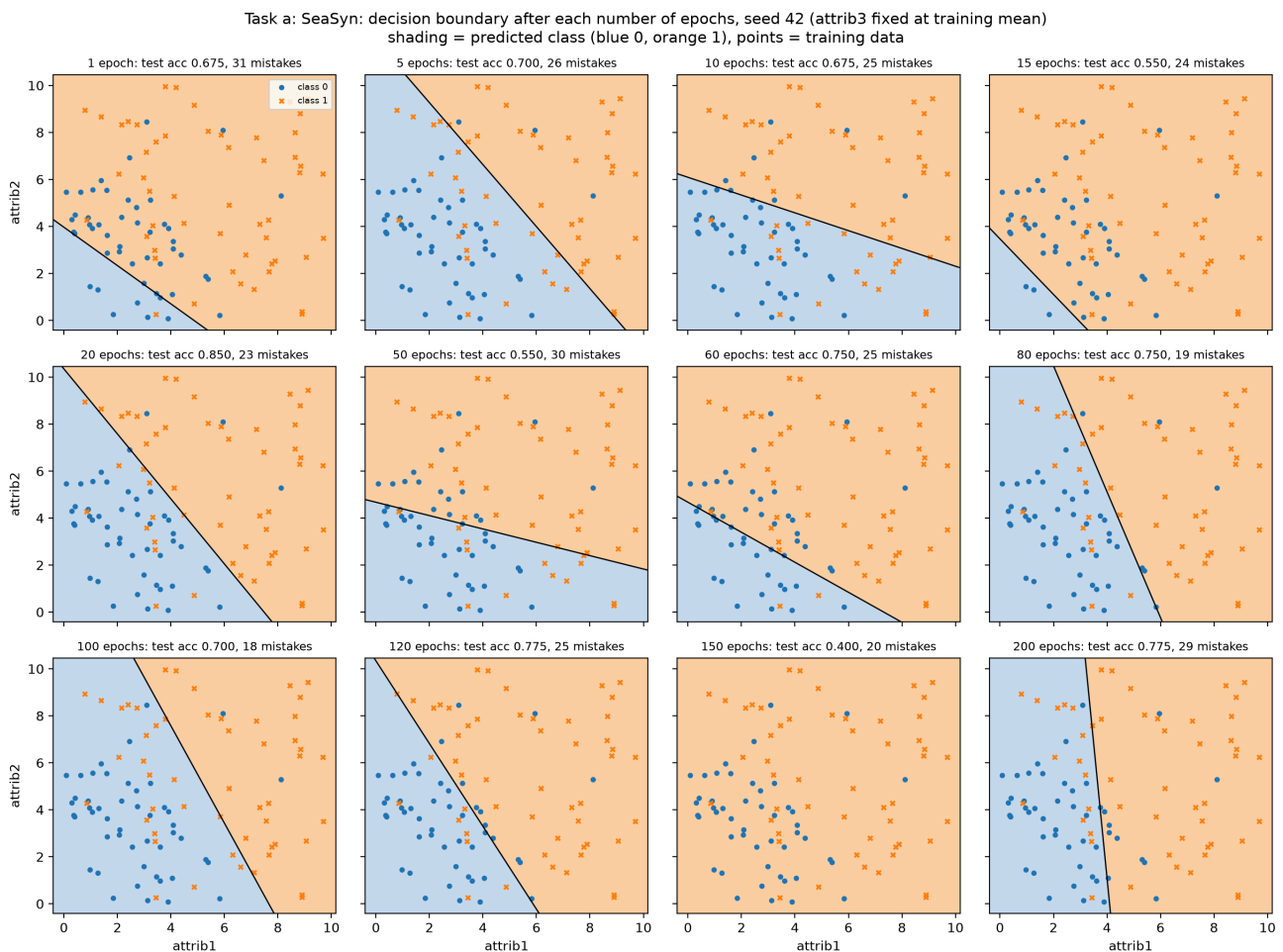


Figure 4: The decision boundary of each model in the first table (seed 42), drawn over the SeaSyn training data. SeaSyn has three features, so each panel is a 2D slice through attrib1 and attrib2 with attrib3 fixed at its training mean. The shading is the predicted class within that slice. A point's real prediction also depends on its own attrib3 value, so a few points can be classified differently from the shading behind them.

How does increasing the number of epochs affect the model's accuracy?

For this perceptron on SeaSyn, training for more epochs does **not** make it more accurate. With seed 42, test accuracy jumps between rows with no upward trend. It is 0.850 after 20 epochs but 0.550 after 50 and 0.400 after 150. Averaged over 10 seeds, it stays roughly flat at 0.69–0.81 for every epoch count, and the standard deviation (up to 0.12) is as large as the differences between rows. Accuracy after 200 epochs (0.747) is no better than after 1 epoch (0.745).

The reason is that SeaSyn is **not perfectly linearly separable**, even though the task treats it as linearly separable data. The classes are roughly split by the line $\text{attrib1} + \text{attrib2} = 8$ (that rule alone gets 88% of the training set right), but about 12% of the points are on the "wrong" side of any line. I checked this separately: a linear-programming feasibility test (scipy) found no line that separates the training set, and a linear SVM (scikit-learn) reached only 85% training accuracy. This is most likely label noise: the SEA concept that this dataset appears to follow (Street & Kim, 2001) deliberately adds noise to about 10% of the class labels. The Perceptron Convergence Theorem only guarantees that the perceptron stops changing when a separating line exists, so it does not apply here. In contrast, the AND self-test, which is separable, converged after 7 epochs.

Instead, some training samples are misclassified in every epoch (between 15 and 36 per epoch across all seeds, never 0, as Figure 3 shows), so every epoch updates the weights and the decision boundary keeps moving. Each update fixes one point but can move the line past many others. The model returned after E epochs is just wherever the last few updates left the line, not the best line found so far. Stopping at a different epoch is like taking a random snapshot, which is why accuracy depends much more on where training stops than on how long it ran. Figure 4 shows this directly. After 20 and 100 epochs the line runs diagonally between the two classes, roughly along the $\text{attrib1} + \text{attrib2} = 8$ split. After 10 and 50 epochs it is much too flat and cuts through the class 1 points, and after 150 epochs it lies outside the slice altogether. The 150-epoch model had put most of its weight on attrib3 , the feature that has no relationship with the class, and it predicts class 1 for 86% of the training set, which explains its 0.400 test accuracy.

The pocket results confirm this. Keeping the best weights seen during training makes test accuracy rise with the number of epochs, from 0.745 to about 0.85 by 15–20 epochs, and then level off with a small standard deviation. Its training accuracy reaches 0.86–0.87, close to the best any line can do. So the perceptron does find good lines during training, but the plain learning rule cannot settle on one. Other common fixes are a learning rate that decreases over time (unlike a constant learning rate, a schedule does change the result even with zero initial weights) or averaging the weights over all updates.

Two caveats apply to these numbers. The test set has only 40 samples, so each mistake changes test accuracy by 2.5 percentage points. Also, as noted in the Data section, the 40 test rows are copies of the first 40 training rows, so test accuracy closely follows training accuracy and says little about how well the model generalises to new data.

Task b: Perceptron on non-linearly separable data (RingSyn)

Task b reuses the Task a perceptron and experiment code without any changes. Only the data files are different (`python3 part2/perceptron.py --task b`). As before, each row is a fresh model trained with seed 42. The "Predicted 1" column is the share of the 300 test samples that the model predicts as class 1.

Model (epoch)	Train accuracy	Test accuracy	Predicted 1	Mistakes in last epoch
Perceptron (1)	0.700	0.743	0.593	133
Perceptron (5)	0.673	0.697	0.440	127
Perceptron (10)	0.764	0.757	0.693	141
Perceptron (15)	0.698	0.727	0.557	140
Perceptron (20)	0.660	0.640	0.537	144
Perceptron (50)	0.689	0.747	0.510	147
Perceptron (60)	0.733	0.790	0.720	134
Perceptron (80)	0.689	0.730	0.580	138
Perceptron (100)	0.669	0.657	1.000	143
Perceptron (120)	0.728	0.773	0.677	129
Perceptron (150)	0.722	0.727	0.570	126
Perceptron (200)	0.711	0.747	0.583	139

The majority-class baseline (always predicting class 1) gets **0.657** on the test set. The confusion matrix of the 200-epoch model on the test set is:

	Predicted 0	Predicted 1
True 0	76	27
True 1	49	148

Mean $\pm$ std test accuracy over seeds 0–9, for the plain and pocket perceptrons (as in Task a):

Epochs	Perceptron test accuracy	Pocket perceptron test accuracy
1	0.723 $\pm$ 0.041	0.723 $\pm$ 0.041
5	0.679 $\pm$ 0.065	0.758 $\pm$ 0.045
10	0.686 $\pm$ 0.058	0.777 $\pm$ 0.036
15	0.699 $\pm$ 0.062	0.775 $\pm$ 0.038
20	0.688 $\pm$ 0.043	0.780 $\pm$ 0.029
50	0.719 $\pm$ 0.040	0.781 $\pm$ 0.010
60	0.689 $\pm$ 0.065	0.783 $\pm$ 0.010
80	0.730 $\pm$ 0.048	0.783 $\pm$ 0.010
100	0.727 $\pm$ 0.068	0.783 $\pm$ 0.010
120	0.713 $\pm$ 0.053	0.782 $\pm$ 0.009
150	0.698 $\pm$ 0.062	0.781 $\pm$ 0.009
200	0.716 $\pm$ 0.058	0.784 $\pm$ 0.009

Task b: RingSyn

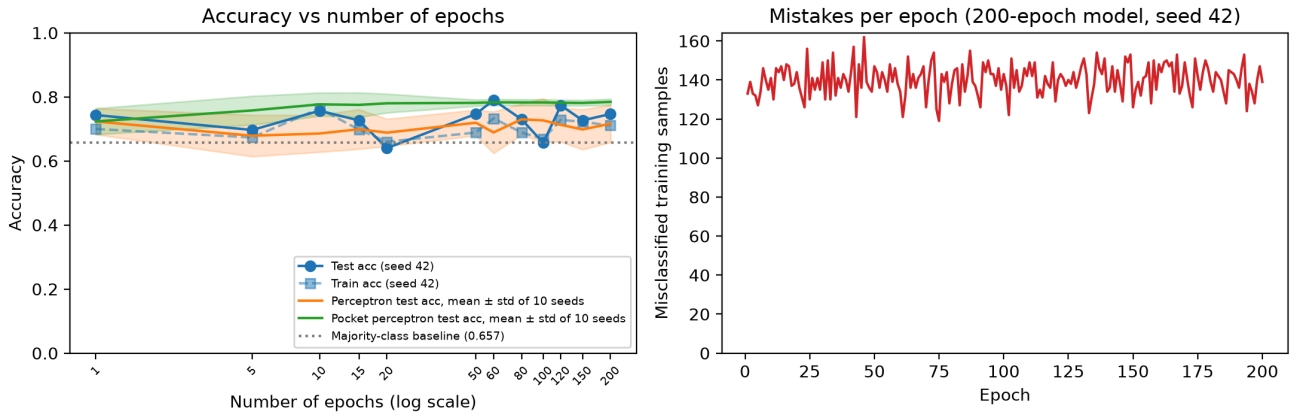


Figure 5: Left: accuracy against the number of epochs (log scale) on RingSyn, with the majority-class baseline as a dotted line. Right: training samples misclassified in each epoch of the 200-epoch model (seed 42), which stays between 119 and 162.

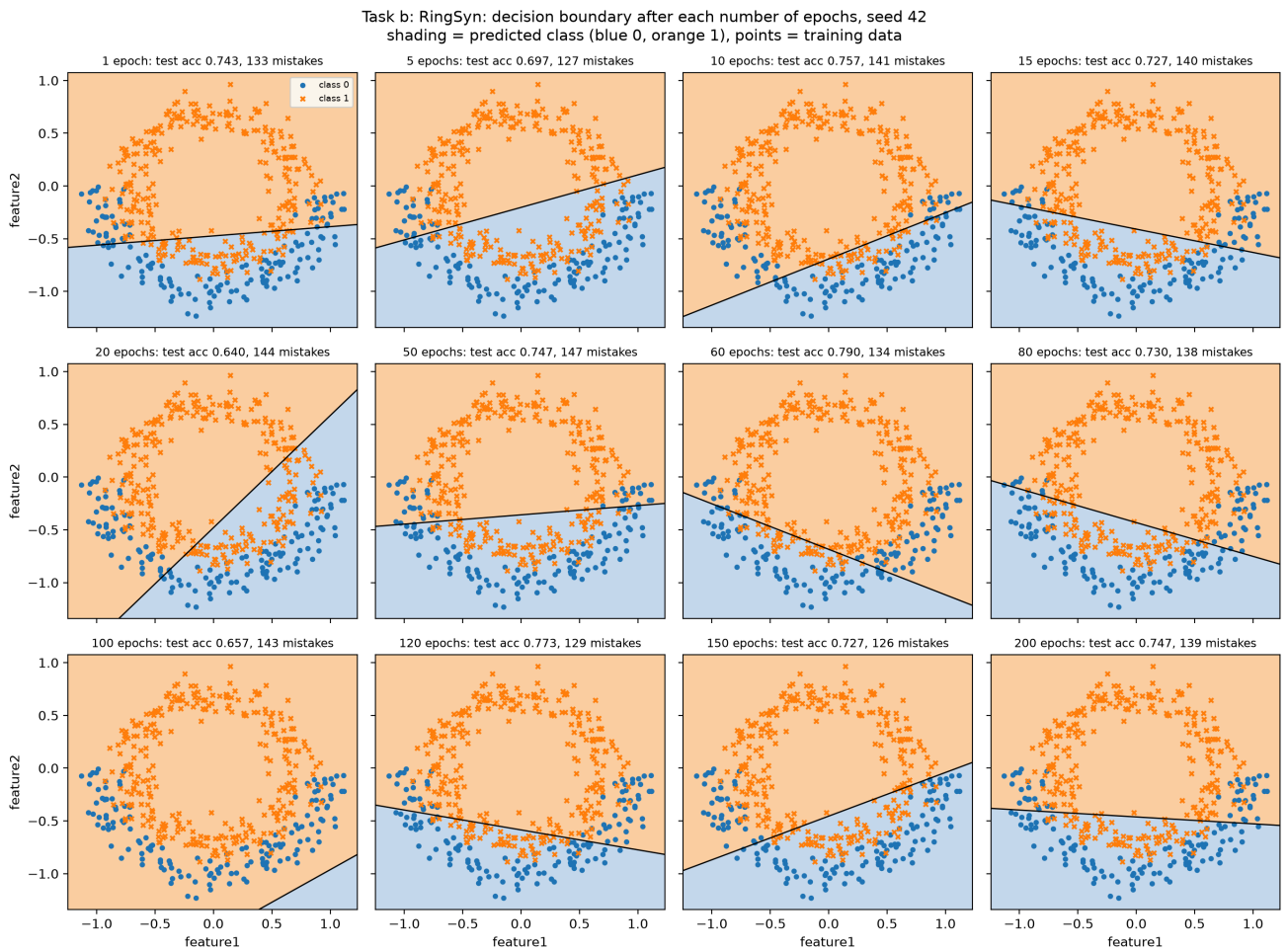


Figure 6: The decision boundary of each model in the first table (seed 42), drawn over the RingSyn training data. RingSyn has only two features, so this is the full picture, not a slice.

How well does the perceptron perform on this dataset?

The perceptron performs **poorly** on RingSyn. Its test accuracy ranges from 0.640 to 0.790 with seed 42, and averages 0.68–0.73 over 10 seeds. This is only 2–7 percentage points better than always predicting class 1 (0.657). The 20-epoch model is actually worse than the baseline. The 100-epoch model predicts

class 1 for every test sample and scores exactly the baseline, 0.657. Training accuracy (0.66–0.76) is no better: always predicting class 1 already gets 0.669 of the training set right.

As in Task a, more epochs do not help. Accuracy jumps between rows with no trend, and the perceptron never converges. In every epoch it misclassifies 119–162 of the 453 training samples, and the count never drops below 107 for any of the 10 seeds. So the weights change throughout training, and each row is just a snapshot of a line that keeps moving.

The reason is the shape of the data. A perceptron can only draw one straight line, but RingSyn's classes are curved: class 1 is a full inner ring, and class 0 is a half-ring outside it, in the lower half only. The best a line can do is what Figure 6 shows in almost every panel: a roughly horizontal line across the lower half, predicting class 1 above it. This gets the top half right (it is all class 1), but in the lower half, the two classes lie on concentric arcs that no straight line can split. The bottom of the inner ring falls below the line and is predicted as class 0, and the two ends of the outer half-ring, which curve up to $\text{feature2} \approx 0$, stay above it and are predicted as class 1. These are the two kinds of error in the confusion matrix: 49 class 1 points predicted as 0, and 27 class 0 points predicted as 1. The perceptron keeps swapping between these errors, so the line keeps tilting and moving up and down.

Even the best line found during training is limited. The pocket version reaches only about 0.78 test accuracy (about 0.79–0.80 on the training set) and levels off after about 20 epochs. This is very different from Task a:

	SeaSyn (Task a)	RingSyn (Task b)
Majority-class baseline (test)	0.525	0.657
Plain perceptron, mean test accuracy over seeds	0.69–0.81	0.68–0.73
Pocket perceptron, mean test accuracy at 200 epochs	0.857	0.784
Gain of the pocket perceptron over the baseline	+33 points	+13 points

Neither dataset converges, but for different reasons. SeaSyn has a good linear boundary, and only noise stops the perceptron from settling on it. RingSyn has no good linear boundary at all, so even the best line is only a modest improvement on guessing the majority class.

The limitation comes from the perceptron's linear decision boundary, not from how it is trained. As an extra check (also printed by `--task b`), I gave the **same, unchanged** perceptron a third input, $r^2 = \text{feature1}^2 + \text{feature2}^2$, the squared distance from the origin. A threshold on r^2 separates the inner ring from the outer half-ring, and a straight line in $(\text{feature1}, \text{feature2}, r^2)$ space corresponds to a circle in the original two features. With this one extra feature, test accuracy rises to 0.79–0.95 across the epoch counts (0.953 after 200 epochs). It still doesn't converge, because the two classes overlap slightly at $\text{radius} \approx 0.85$. Choosing a feature like r^2 by hand only works because I looked at the data. A multilayer perceptron's hidden layer can learn suitable non-linear features by itself, which is what Task c tests.

Task c: MLP on non-linearly separable data (RingSyn)

Setup. `part2/mlp.py` uses scikit-learn's `MLPClassifier` with **one hidden layer of 8 neurons** and the **tanh** activation, trained with the default **adam** optimiser (`max_iter=2000`, `random_state=42`). As for the perceptron, both features are standardised using the training set's mean and standard deviation, inside a scikit-learn pipeline, so the test data is never used for fitting. I chose 8 neurons, in the middle of the suggested 4–10, so the network has enough neurons to bend its boundary around the inner ring. I chose tanh because it is smooth, which suits a curved boundary. Training stopped after 1113 epochs, once the

training loss had stopped improving, with no convergence warning. While setting this up I briefly tried 4, 8 and 10 hidden neurons with both tanh and ReLU. All gave a similar mean test accuracy (0.92–0.94 over 10 seeds), except ReLU with 4 neurons, which was less stable between seeds. As the handout asks for one reasonable setup, only this one is reported below. It uses the same train/test files as Task b (`python3 part2/mlp.py`).

Model	Train accuracy	Test accuracy
Majority class (always predict 1)	0.669	0.657
MLP (8 hidden neurons, tanh)	0.927	0.937

Test confusion matrix of the MLP:

	Predicted 0	Predicted 1
True 0	93	10
True 1	9	188

To check that 0.937 was not a lucky initialisation, the same setup was also trained with `random_state` 0–9. Test accuracy was 0.936 ± 0.003 (min 0.930, max 0.943), and training accuracy was 0.926 ± 0.003 .

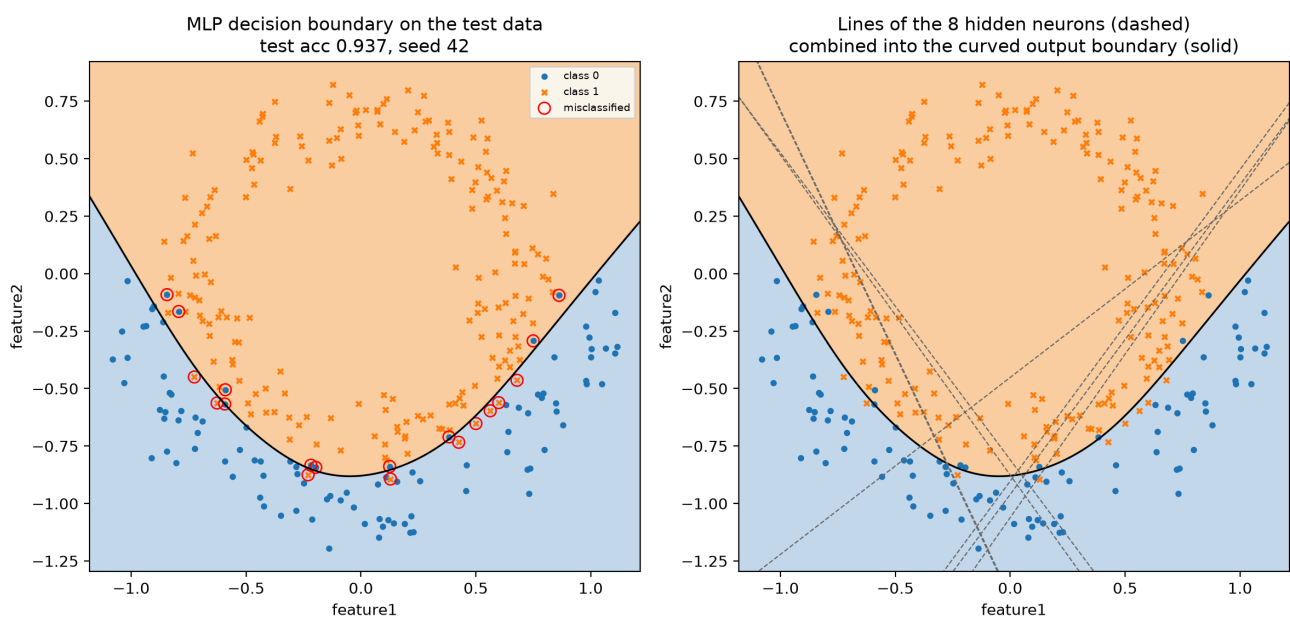


Figure 7: Left: the MLP's predicted regions (blue = class 0, orange = class 1) over the RingSyn test data, with the 19 misclassified test points circled in red. Right: the same boundary (solid) with the line $w \cdot x + b = 0$ of each of the 8 hidden neurons (dashed).

Epoch sweep. To compare the MLP with the perceptron on the same terms, `mlp.py` also repeats the Task b experiment for the MLP. For each number of epochs in the handout's list, a fresh MLP with the same setup is trained for exactly that many epochs (`max_iter`). With the same seed, the initial weights and shuffle order are identical, so each row is a snapshot of the main model partway through training. I checked this by confirming that the 200-epoch model's loss curve matches the first 200 epochs of the full run. The last column is the mean $\pm$ std test accuracy over seeds 0–9.

Model (epoch)	Train accuracy	Test accuracy	Predicted 1	Test accuracy, 10 seeds
MLP (1)	0.667	0.673	0.330	0.579 $\pm$ 0.133
MLP (5)	0.675	0.677	0.333	0.598 $\pm$ 0.127
MLP (10)	0.689	0.700	0.357	0.624 $\pm$ 0.112
MLP (15)	0.693	0.730	0.400	0.653 $\pm$ 0.094
MLP (20)	0.700	0.737	0.440	0.671 $\pm$ 0.089
MLP (50)	0.709	0.767	0.537	0.739 $\pm$ 0.068
MLP (60)	0.724	0.760	0.570	0.758 $\pm$ 0.056
MLP (80)	0.748	0.780	0.630	0.763 $\pm$ 0.057
MLP (100)	0.748	0.787	0.670	0.775 $\pm$ 0.043
MLP (120)	0.744	0.807	0.703	0.781 $\pm$ 0.043
MLP (150)	0.781	0.813	0.723	0.807 $\pm$ 0.017
MLP (200)	0.808	0.833	0.730	0.835 $\pm$ 0.017
MLP (1113, fully trained)	0.927	0.937	0.660	0.936 $\pm$ 0.003

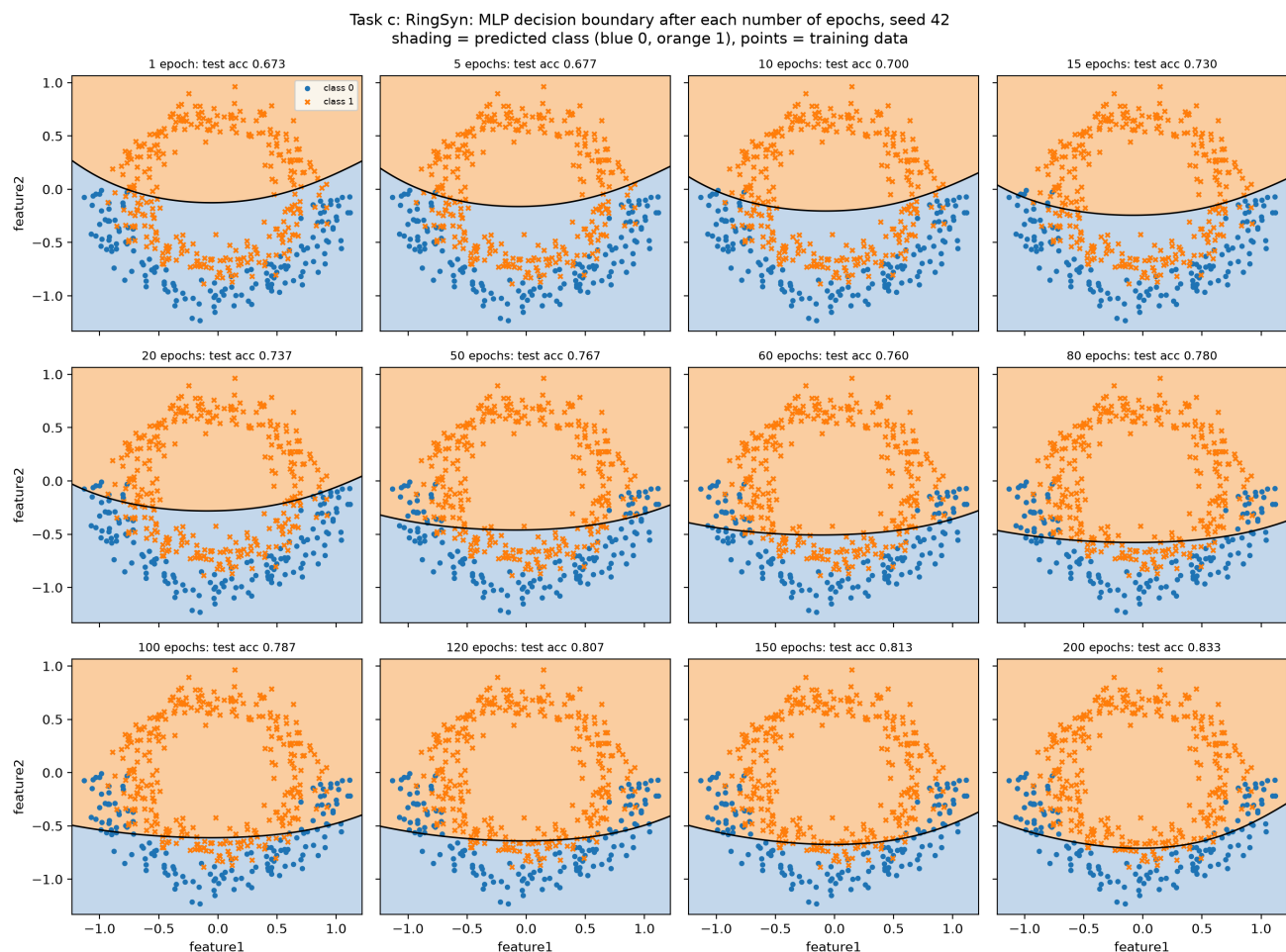


Figure 8: The MLP's decision boundary after each number of epochs (seed 42), drawn over the RingSyn training data, in the same layout as Figure 6 for the perceptron.

How does the performance of the MLP compare to the Perceptron from Task b on this dataset?

The MLP is much better. Its test accuracy is 0.937, compared with:

- 0.640–0.790 for the perceptron in Task b with seed 42 (0.747 after 200 epochs), and 0.68–0.73 on average over 10 seeds,
- 0.784 for the pocket perceptron, which is roughly the best a single straight line can do on this data,
- 0.657 for the majority-class baseline.

So the MLP is about 19 percentage points better than the perceptron after 200 epochs, 15 points better than the perceptron's best row, 15 points better than the best line, and 28 points better than the baseline. It makes 19 test mistakes instead of the perceptron's 76 (200-epoch model), and its mistakes are balanced across both classes (10 and 9) instead of mostly being class 1 points predicted as 0. It is also far more stable: its test accuracy varies by ± 0.003 between random seeds, compared with about ± 0.06 for the perceptron between seeds and up to 15 points between epoch counts.

The epoch sweep shows that the two models also learn very differently. The perceptron's accuracy jumps up and down from one epoch count to the next (0.640–0.790), because each mistake moves its line by a full step. The MLP's accuracy rises almost every row, from 0.673 after 1 epoch to 0.833 after 200, and its spread between seeds shrinks from ± 0.133 to ± 0.017 . The MLP is trained by gradient descent on a smooth loss, and adam takes many small steps that each reduce the loss a little, so the boundary moves gradually in one direction instead of jumping around. Figure 8 shows this: the boundary starts as an almost flat curve, much like the perceptron's line, slides steadily down towards the gap between the classes, and only starts to bend into a U at around 200 epochs.

This also means the MLP needs more training than the perceptron. For short training (up to about 20 epochs) it is actually no better than the perceptron, and after the handout's longest run of 200 epochs it reaches 0.833, about 5 points better than the best line (0.784). It only reaches 0.937 when it is allowed to train until its loss stops improving, after 1113 epochs. Its advantage comes from what it can represent, a curved boundary, but it takes longer to find that boundary than the perceptron takes to find a line.

Was the MLP able to better capture the structure of the data? Why or why not?

Yes. Figure 7 shows that the MLP's boundary is a smooth U-shaped curve that follows the gap between the bottom of the inner ring (class 1) and the outer half-ring (class 0). The perceptron could only place a straight line across this gap (Figure 6). The MLP does not draw a full circle, because it does not need to: there are no class 0 points above $\text{feature2} \approx 0$, so it leaves the top of its class 1 region open. It has learned the part of the structure that actually separates the two classes.

It can do this because of the hidden layer and its non-linear activation:

1. **Each hidden neuron is like a perceptron.** It computes its own weighted sum $w \cdot x + b$, so it has its own straight line (the dashed lines in Figure 7). The lines fall into two groups. One group is steep and runs along the left side of the U, and the other runs along the right side.
2. **The tanh activation makes each neuron's output non-linear.** Each neuron's output changes smoothly from -1 to $+1$ as a point crosses its line, instead of growing without limit. Each neuron effectively answers "which side of my line is this point on?", with a soft transition near the line.
3. **The output neuron combines these answers.** It takes a weighted sum of the 8 hidden outputs, so it can say "class 1 if the point is on the inner side of the left lines and the inner side of the right lines". Combining several soft half-planes like this gives a region whose boundary bends, and because tanh is smooth, the corners where lines meet are rounded into the curve in Figure 7.

In other words, the hidden layer learns a new set of features in which the classes are almost linearly separable, and the output neuron then draws a linear boundary in that new space. This is the same idea as the r^2 feature at the end of Task b, but the MLP learned its features from the data instead of me choosing

them by looking at the data. The non-linear activation is essential: without it, the output would be a linear function of a linear function of the input, which is still linear, so the MLP could do no better than the perceptron.

The MLP is not perfect. All 19 misclassified test points are between 0.78 and 0.90 from the origin, while the test points as a whole range from 0.41 to 1.21. This is exactly the band where the inner ring and the outer half-ring overlap (Data section). Figure 7 shows these points on both sides of the boundary, mixed in with correctly classified points of the other class, so no smooth boundary could get all of them right. Training accuracy (0.927) is close to test accuracy (0.937), so the network is not overfitting. The remaining errors come from the overlap in the data, not from a lack of flexibility in the model.

References / External resources

Part 1

- The Q1–Q5 answers were worked out by hand from the problem definition in the assignment handout. No external code was used.
- The optional checking script `part1/gantt.py` is my own code. It uses Python 3 and the Matplotlib library (J. D. Hunter, "Matplotlib: A 2D Graphics Environment", *Computing in Science & Engineering*, 9(3), 90–95, 2007) to draw Figures 1 and 2.

Part 2

- The perceptron in `part2/perceptron.py` is implemented from scratch with numpy (C. R. Harris et al., "Array programming with NumPy", *Nature*, 585, 357–362, 2020), following the perceptron learning rule (F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain", *Psychological Review*, 65(6), 386–408, 1958). No machine learning library is used for the perceptron itself.
- The "pocket" comparison in Tasks a and b is based on the pocket algorithm (S. I. Gallant, "Perceptron-based learning algorithms", *IEEE Transactions on Neural Networks*, 1(2), 179–191, 1990).
- The MLP in `part2/mlp.py` uses scikit-learn's `MLPClassifier`, `StandardScaler`, `make_pipeline`, `accuracy_score` and `confusion_matrix` (F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, 12, 2825–2830, 2011). It is trained with the Adam optimiser (D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization", *ICLR*, 2015).
- Figures 3–8 were drawn with Matplotlib (reference above).
- The linear separability check for SeaSyn in Task a was run separately from the submitted programs, using SciPy's `linprog` (P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python", *Nature Methods*, 17, 261–272, 2020) and scikit-learn's `LinearSVC`.
- The SEA concept mentioned in Task a is from W. N. Street and Y. Kim, "A Streaming Ensemble Algorithm (SEA) for Large-Scale Classification", *Proceedings of the 7th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 377–382, 2001.

Use of AI assistance (Part 2)

- I used Claude (Anthropic), an AI assistant, through Claude Code for Part 2. It wrote most of the code in `part2/perceptron.py` and `part2/mlp.py`, ran the experiments and checks, and drafted most of the Part 2 text and tables in this report from the program output.
- It also carried out the data checks that shaped the discussion: that SeaSyn's test rows are copies of its first 40 training rows, that SeaSyn is not perfectly linearly separable, and the shape of the two

RingSyn classes.

- All results, tables and figures in Part 2 were produced by running the submitted programs, and can be reproduced with the commands in `readme.txt`.